1. (3 Punkte) Die Klasse Ding hat die Attribute name und wert. Implementiere die Klasse Ding. Folgender Dialog soll möglich sein:

```
d = Ding("A",100)
print(d)
```

```
Das Ding A hat den Wert 100
```

Lösung:

```
class Ding:
    def __init__(self,name,wert):
        self.name = name
        self.wert = wert

    def __str__(self):
        return f"Das Ding {self.name} hat den Wert {self.wert}"
```

2. (3 Punkte) Die Klasse Zutat hat die Attribute bezeichnung und menge. Implementiere die Klasse Zutat. Folgender Dialog soll möglich sein:

```
z = Zutat(800,"Mehl")
print(z)
```

```
Es werden 800g Mehl benötigt.
```

Lösung:

```
class Zutat:
    def __init__(self,menge,bezeichnung):
        self.menge = menge
        self.bezeichnung = bezeichnung

    def __str__(self):
        return f"Es werden {self.menge}g {self.bezeichnung} benötigt."
```

3. (3 Punkte) Schreibe Code, der den folgenden Dialog möglich macht:

```
a = A('H')
b = A('C')
print(a.aba, a.d)
print(b.aba, b.d)
```

```
17 H
17 C
```

Lösung:

```
class A:
    def __init__(self, d):
        self.d = d
        self.aba = 17
```

4. (3 Punkte) Schreibe Code, der den folgenden Dialog möglich macht:

```
k1 = K(1)
k2 = K(2)
print(k1.x, k1.y, k2.x, k2.y)

1 1 2 2
```

Lösung:

```
class K:
    def __init__(self, x):
        self.x = x
        self.y = x
```

5. (5 Punkte) Die Klasse Parkhaus hat die Attribute plaetze (Anzahl aller Plätze) und belegt (Anzahl der belegten Plätze). Dem Konstruktor wird die Anzahl der Plätze übergeben, zu Beginn ist kein Platz belegt.

   - `frei()` gibt die Anzahl der freien Plätze zurück.
   - `einfahren()` belegt einen Platz und gibt True zurück. Ist kein Platz frei, wird False zurückgegeben.
   - `ausfahren()` gibt einen Platz wieder frei, wenn mindestens ein Platz belegt ist.

   Implementiere die Klasse Parkhaus. Folgender Dialog soll möglich sein:

```
p = Parkhaus(2)
print(p.einfahren(), p.einfahren(), p.einfahren())
print(p.frei())
p.ausfahren()
print(p)

True True False
0
1 von 2 Plätzen belegt
```

Lösung:

```
class Parkhaus:
    def __init__(self, plaetze):
        self.plaetze = plaetze
        self.belegt = 0

    def frei(self):
        return self.plaetze - self.belegt

    def einfahren(self):
        if self.frei() > 0:
            self.belegt += 1
            return True
        return False

    def ausfahren(self):
        if self.belegt > 0:
            self.belegt -= 1

    def __str__(self):
        return f'{self.belegt} von {self.plaetze} Plätzen belegt'
```

6. (3 Punkte) Zeichne die Objektdiagramme für die Objekte p und q nach dem Durchlaufen des Codes.

```
class Punkt:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def verschieben(self, dx):
        self.x += dx

p = Punkt(2, 3)
q = Punkt(0, 1)
q.verschieben(4)
p.y = q.x
```

Lösung:

p: Punkt
x = 2
y = 4

q: Punkt
x = 4
y = 1

7. (6 Punkte) a. Gib für die Klasse ein passendes Klassendiagramm an.

   b. Was erscheint auf der Konsole?

```
class K:
    def __init__(self,s):
        self.k1 = 1
        self.b1 = False
        self.s = s

    def m1(self, b2):
        if b2:
            return self.s + '#'
        else:
            return self.s + '?'

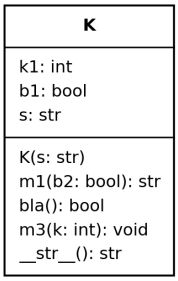
    def bla(self):
        return not self.b1

    def m3(self, k):
        self.k1 += k
        self.b1 = not self.b1

    def __str__(self):
        return str(self.k1) + ' ' + str(self.b1) + ' ' + self.s

k = K('hi')
k.m3(4)
temp = k.m1(k.bla())
k.s = temp
print(k)
```

Lösung:



b. 5 True hi?

8. (8 Punkte) a. Implementiere eine Klasse `Buch` mit den Attributen `autor` und `titel`.
b. Erzeuge zwei Buch-Objekte `b1` und `b2`: *Homo faber* von Max Frisch und *Buddenbrooks* von Thomas Mann.
c. Erstelle eine Liste `buecher` die `b1` und `b2` enthält.
d. Erstelle ein Dictionary `buchmap` mit dem Titel eines Buches als key und dem Buch-Objekt als value. `buchmap` soll zu Beginn Einträge für `b1` und `b2` enthalten.
e. Erweitere `buchmap` um einen Eintrag: *Agnes* von Peter Stamm.
f. In `buchmap` seien noch mehrere Bücher eingefügt worden. Welcher Ausdruck mit `buchmap` liefert den Autor von `Der Prozess` (wir nehmen an, dass das entsprechende Buch in `buchmap` vorhanden ist).

Lösung:

```
class Buch:
    def __init__(self,autor,titel):
        self.autor = autor
        self.titel = titel

b1 = Buch("Max Frisch","Homo faber")
b2 = Buch("Thomas Mann","Buddenbrooks")
buecher = [b1, b2]
buchmap = {"Homo faber":b1, "Buddenbrooks":b2}
buchmap["Agnes"] = Buch("Peter Stamm","Agnes")
buchmap["Der Prozess"].autor
```

9. (9 Punkte) a. Implementiere eine Klasse `Filmmusik` mit den Attributen `film` und `komponist`, so dass die folgenden Filmmusik-Objekte erzeugt werden können:

```
m1 = Filmmusik("Jurassic Park","John Williams")
m2 = Filmmusik("Star Wars","John Williams")
m3 = Filmmusik("Terminal","John Williams")
m4 = Filmmusik("Inception","Hans Zimmer")
m5 = Filmmusik("Interstellar","Hans Zimmer")
m6 = Filmmusik("Lord of the Rings","Howard Shore")
```

b. Implementiere eine string-Methode, die mit `print(m1)` folgende Ausgabe erzeugt:
`Jurassic Park - Musik: John Williams`
c. Erzeuge eine Liste `fmlist` mit den Objekten m1, m2... m6. Die Objekte m1 bis m6 können als erzeugt vorausgesetzt werden.
d. Gehe mit einer Schleife durch die `fmlist` und gib alle Filmtitel mit Musik von John Williams aus.
e. Erzeuge für die 6 Filme in `fmlist` ein Dictionary `fmmap`, bei dem jedem Komponistennamen (key) eine Liste seiner Filmmusik-Objekte als value zugeordnet ist.

Beispiel einer Nutzung von `fmmap`:

```
>>> for f in fmmap["Hans Zimmer"]:
    print(f)

Inception - Musik: Hans Zimmer
Interstellar - Musik: Hans Zimmer
```

Lösung:

```
class Filmmusik:
    def __init__(self,film,komponist):
        self.film = film
        self.komponist = komponist

    def __str__(self):
        return self.film + " - Musik: " + self.komponist


fmlist = [m1,m2,m3,m4,m5,m6]
for f in fmlist:
    if f.komponist == "John Williams":
        print(f.film)

fmmap =  {"John Williams":[m1,m2,m3],
          "Hans Zimmer":[m4,m5],"Howard Shore":[m6]}
```

10. (4 Punkte) Welche Ausgabe erscheint jeweils auf der Konsole?

```
a.
class Clock:
    def __init__(self, time):
        self.time = time
    def print_time(self):
        time = '6:30'
        print(self.time)

clock = Clock('5:30')
clock.print_time()

b.
class Clock:
    def __init__(self, time):
        self.time = time
    def print_time(self, time):
        print(time)

clock = Clock('5:30')
clock.print_time('10:30')

c.
class Clock:
    def __init__(self, time):
        self.time = time
    def print_time(self):
        print(self.time)

boston_clock = Clock('5:30')
paris_clock = boston_clock
paris_clock.time = '10:30'
boston_clock.print_time()
```

Lösung:

a. 5:30
b. 10:30
c. 10:30

11. (4 Punkte) Was erscheint auf der Konsole?

```
class Konto:
    def __init__(self, stand):
        self.stand = stand

def einzahlen(konto, betrag):
    konto.stand += betrag
    betrag = 0

a = Konto(100)
b = a
c = Konto(150)
x = 50
einzahlen(b, x)
print(a.stand, b.stand, c.stand, x)
print(a == b, a == c, a.stand == c.stand)
```

Lösung:

```
150 150 150 50
True False True
```

a und b verweisen auf dasselbe Objekt, c ist ein anderes Objekt mit demselben Kontostand. Die Zuweisung an den Parameter betrag ändert x nicht.

12. (3 Punkte) Die Klasse Konto hat das Attribut kontostand. Beim Erstellen eines Kontos wird der Wert von Kontostand auf 0 gesetzt. Implementiere die Klasse Konto unter Wahrung des Geheimnisprinzips mit getter und setter-Methode.

Lösung:

```
class Konto:
    def __init__(self):
        self.__kontostand = 0

    def set_kontostand(self,x):
        self.__kontostand = x

    def get_kontostand(self):
        return self.__kontostand
```

13. (3 Punkte) Die Klasse Person hat das Attribut alter. Dem Konstruktor von Person wird der Wert von alter übergeben. Implementiere die Klasse Person unter Wahrung des Geheimnisprinzips mit getter und setter-Methode, wobei allen Unterklassen von Person der direkte Zugriff auf das Attribut alter möglich sein soll.

Lösung:

```
class Person:
    def __init__(self, alter):
        self._alter = alter

    def set_alter(self,x):
        self._alter = x

    def get_alter(self):
        return self._alter
```

14. (5 Punkte) Die Klasse Temperatur hat das Attribut celsius. Dem Konstruktor wird der Wert von celsius übergeben. Implementiere die Klasse Temperatur unter Wahrung des Geheimnisprinzips mit getter- und setter-Methode. Der Setter lehnt Werte unter -273.15 ab, das Attribut behält dann seinen Wert. Auch der Konstruktor benutzt den Setter, der Startwert des Attributs ist 0. Folgender Dialog soll möglich sein:

```
t = Temperatur(20)
t.set_celsius(-300)
print(t.get_celsius())
t.set_celsius(-10)
print(t.get_celsius())
u = Temperatur(-500)
print(u.get_celsius())

20
-10
0
```

Lösung:

```
class Temperatur:
    def __init__(self, celsius):
        self.__celsius = 0
        self.set_celsius(celsius)

    def get_celsius(self):
        return self.__celsius

    def set_celsius(self, celsius):
        if celsius >= -273.15:
            self.__celsius = celsius
```

15. (3 Punkte) Was erscheint auf der Konsole? Begründe.

```
class Box:
    def __init__(self, wert):
        self.__wert = wert

    def get_wert(self):
        return self.__wert

b = Box(3)
b.__wert = 8
print(b.get_wert())
print(b.__wert)
c = Box(5)
print(c.__wert)
```

Lösung:

3
8
AttributeError

`b.__wert = 8` legt von außen ein neues öffentliches Attribut an, das private Attribut bleibt 3. Bei c gibt es dieses neue Attribut nicht, von außen ist das private Attribut nicht erreichbar.

16. (6 Punkte) a. Gib für die beiden Klassen ein passendes Klassendiagramm an.

    b. Was erscheint auf der Konsole?

```
class L:
    def __init__(self,a,d):
        self.a = a
        self.d = d

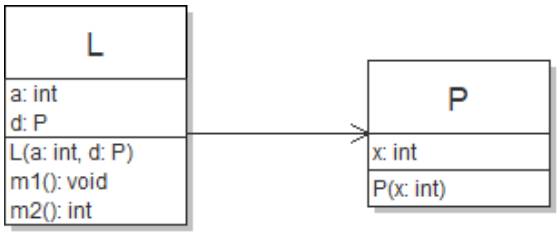
    def m1(self):
        self.a = self.d.x + 1

    def m2(self):
        return self.a

class P:
    def __init__(self,x):
        self.x = x

p = P(1)
k = L(2,p)
print(k.m1())
print(k.m2())
```

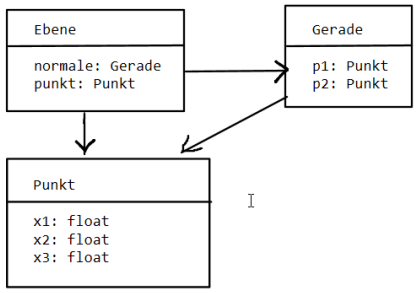
Lösung:



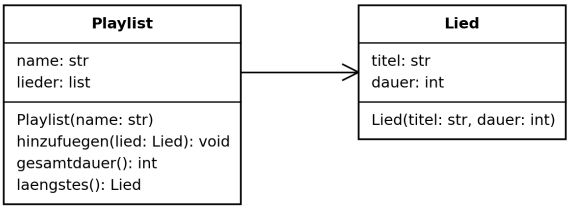
b. None, 2

17. (3 Punkte) Es sollen Objekte aus dem Bereich der Vektorgeometrie modelliert werden. Eine Ebene, eine Gerade und ein Punkt. Ein Punkt soll durch drei float-Werte `x1, x2, x3` festgelegt sein. Eine Gerade soll durch zwei Punkte bestimmt sein, eine Ebene soll durch eine Gerade (ihre Normale) und einen Punkt definiert sein. Zeichne ein passendes Klassendiagramm mit den Klassennamen und den Attributen.

Lösung:



18. (6 Punkte) Implementiere die beiden Klassen. `laengstes()` gibt das Lied mit der größten Dauer zurück.



Folgender Dialog soll möglich sein:

```
p = Playlist('Sport')
p.hinzufuegen(Lied('Thunderstruck', 292))
p.hinzufuegen(Lied('Eye of the Tiger', 246))
print(p.gesamtdauer())
print(p.laengstes().titel)

538
Thunderstruck
```

Lösung:

```
class Lied:
    def __init__(self, titel, dauer):
        self.titel = titel
        self.dauer = dauer

class Playlist:
    def __init__(self, name):
        self.name = name
        self.lieder = []

    def hinzufuegen(self, lied):
        self.lieder.append(lied)

    def gesamtdauer(self):
        summe = 0
        for lied in self.lieder:
            summe += lied.dauer
        return summe

    def laengstes(self):
        bestes = self.lieder[0]
        for lied in self.lieder:
            if lied.dauer > bestes.dauer:
                bestes = lied
        return bestes
```

19. (5 Punkte) Ein Ordner kann in einem anderen Ordner liegen.

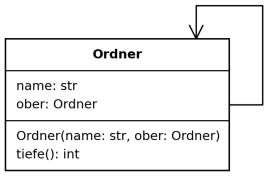
```
class Ordner:
    def __init__(self, name, ober=None):
        self.name = name
        self.ober = ober

c = Ordner('C:')
s = Ordner('Schule', c)
i = Ordner('Informatik', s)
```

(a) Zeichne das Klassendiagramm.

(b) Ergänze die Methode `tiefe()`, die zurückgibt, in wie vielen Ordnern ein Ordner liegt. `i.tiefe()` ergibt 2, `c.tiefe()` ergibt 0.

Lösung:




```
    def tiefe(self):
        n = 0
        o = self.ober
        while o is not None:
            n += 1
            o = o.ober
        return n
```

20. (4 Punkte) Die Klasse A hat ein Attribut zahl. Dem Konstruktor von A wird der Wert für das Attribut zahl übergeben. Die Klasse B erbt von A und erhält ein weiteres Attribut zahl2. Dem Konstruktor von B werden Werte für beide Attribute übergeben, der Konstruktor von B nutzt den Konstruktor von A. Implementiere die Klasse B.



Lösung:

```
class B(A):
    def __init__(self,zahl,zahl2):
        super().__init__(zahl)
        self.zahl2 = zahl2
```

21. (3 Punkte) Die Klasse A hat eine Methode berechne, die einen int-Wert zurückgibt. Die Klasse B erbt von A und überschreibt die Methode berechne. Sie addiert noch 10 auf das Ergebnis von A drauf. Implementiere die Methode berechne in der Klasse B.

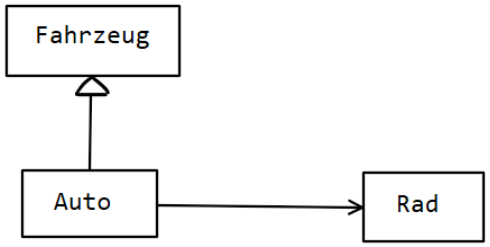


Lösung:

```
class B(A):
    def berechne(self):
        return super().berechne() + 10
```

22. (3 Punkte) Ein Auto ist ein Fahrzeug und hat Räder. Zeichne ein Klassendiagramm mit den Klassen Fahrzeug, Auto, Rad.

Lösung:



23. (8 Punkte) Was erscheint auf der Konsole?

```
class Spell:
    def __init__(self, incantation, name):
        self.name = name
        self.incantation = incantation
    def __str__(self):
        return self.name+' '+self.incantation+'\n'+self.getDescription()
    def getDescription(self):
        return 'No description'
    def execute(self):
        print(self.incantation)

class Accio(Spell):
    def __init__(self):
        super().__init__('Accio', 'Summoning Charm')

class Confundo(Spell):
    def __init__(self):
        super().__init__('Confundo', 'Confundus Charm')
    def getDescription(self):
        return 'Causes the victim to become confused and befuddled.'

def studySpell(spell):
    print(spell)

spell = Accio()
spell.execute()
studySpell(spell)
studySpell(Confundo())
```

Lösung:

```
Accio
Summoning Charm Accio
No description
Confundus Charm Confundo
Causes the victim to become confused and befuddled.
```

24. (4 Punkte) Erkläre in ein bis zwei Sätzen.
    (a) Was ist der Unterschied zwischen einer Klasse und einem Objekt?
    (b) Welche Aufgabe hat der Konstruktor?
    (c) Was bedeutet Geheimnisprinzip?
    (d) Was ist der Unterschied zwischen einer Assoziation und einer Vererbung?

Lösung: a) Eine Klasse ist der Bauplan, ein Objekt ist eine nach diesem Bauplan erzeugte Instanz.
b) Er wird beim Erzeugen eines Objekts durchlaufen und versetzt es in einen vernünftigen Anfangszustand.
c) Die Attribute sind nach außen nicht direkt sichtbar, der Zugriff läuft über Methoden. Wie die Klasse ihre Daten speichert, bleibt verborgen.
d) Assoziation: „hat ein“, ein Attribut verweist auf ein Objekt einer anderen Klasse. Vererbung: „ist ein“, die Unterklasse übernimmt die Attribute und Methoden der Oberklasse.